

Graph Neural Networks for Power Resource Allocation in NOMA Wireless Networks

Ryan Himes

SEAS

University of Pennsylvania

Philadelphia, PA USA

Abstract—Optimizing the allocation of power to different users in non-orthogonal multiple access (NOMA) systems is important to both maximize throughput and prevent signal interference when using the same frequency bands. This optimization problem has been shown to be both non-convex and NP-hard [1], so currently there is no known algorithm to find the best optimization parameters for every scenario. This has caused current research to instead investigate how to estimate the best optimization parameters. The commonly used technique was to use the weighted minimum mean-squared error (WMMSE) algorithm to estimate power allocation. The three reviewed papers attempt to improve on the WMMSE algorithm by either improving the average sum-rate, quality of service (QoS) probability, or speed using graph neural networks.

Keywords—power resource allocation, graph neural networks, NOMA networks, WMMSE

I. Introduction

Non-orthogonal multiple access (NOMA) systems can achieve a higher throughput compared to systems that force the signals to be orthogonal since signals can be sent on the same frequencies. In order to achieve this, signals must still be able to be decoded by the end user. Some signals on the same frequencies may become weaker compared to others when reaching the user because of issues such as physical obstacles, distance, or using different hardware. To ensure stronger signals don't overpower weaker ones, more power needs to be allocated to signals with more interference between the sender and receiver.

A NOMA system has a maximum amount of power it can use to allocate to different signals being transmitted to achieve different system objectives. Some of these objectives include maximizing average sum rate and minimizing the quality of service (QoS) violation probability [2]. The power allocation optimization problem can be represented in Equation 1[2] to maximize the average sum rate while minimizing the QoS violation probability where P is the power vector and Υ_{min}

is the minimum required rate for each link to not break the QoS agreement.

$$\begin{aligned} \max_P \frac{1}{N} \sum_{i=1}^N \Upsilon_i \\ \Upsilon_i \geq \Upsilon_{min} \\ 0 \leq p_i \leq p_{max} \end{aligned} \tag{1}$$

The optimization problem has been shown to be non-convex so the optimal solution exists at a global minimum while other local minimum values exist. The problem is also NP-hard so there is currently no way of finding the best possible solution for every scenario. In order to estimate the solution, different optimization techniques have been employed. One of the main techniques used is the weighted minimum mean-squared error (WMMSE) algorithm. WMMSE has two main drawbacks, it can only find the local minimum error and it does not scale well with large networks, scaling at a quadratic rate, slowing down real time use [1]. In the context of a non-convex problem only finding the local minimum leaves room for improvement to find the global minimum error. The logical next step to solve these two downsides is to use multi-layer perceptrons (MLPs) which can find the global minimum error of an optimization problem and the time complexity of a single MLP scales linearly with the input size.

Even though MLPs are able to find the global minimum error as well as have a low time complexity, MLPs are not suited to accurately represent a wireless network topology, since a classic MLP takes a vector as input, and can not generalize well outside of the training data used. This causes MLPs to work well on power allocation optimization on small networks but cannot scale to larger networks. In addition, because of the unstructured nature of the input data, more training samples are needed to train the network [3]. To overcome these two weaknesses in MLPs, graph neural networks (GNNs) can be used to structure data and

can generalize better through the nature of message passing [2].

A GNN problem can be best described as first constructing a graph then running a message passing algorithm on the created graph. Constructing the graph involves creating vertices that each contain values with related vertices connected by either a directed or undirected edge. In some GNN architectures, edges between vertices can also hold values. Message passing is used to combine vertex values throughout the graph after many steps. Each message passing step, which is ran on every vertex, can be broken into two steps, aggregate then update. For each vertex, aggregate combines all the values in each neighboring node then the update step combines the aggregated value with the vertex value to update the vertex value. These two steps contain trainable parameters which allows a GNN to train, the parameters can take many different forms which leads to varying GNN architectures. The resulting graph from all the message passing steps can be used to make different predictions such as using individual vertex values to make vertex level predictions or all vertex values to make graph level predictions.

Each of the reviewed papers modify the GNN architecture in different ways to find the optimal solution for power allocation or optimize current solutions for quicker real time predictions. Joint wireless power control and spectrum allocation combines power allocation with resource block allocation [2]. Unfolding WMMSE modifies the existing WMMSE algorithm using GNNs to decrease the prediction time [4]. Air MPNN and Air MPRNN reduce the overhead needed when predicting power allocation using GNNs [5].

II. Joint Wireless Power Control and Spectrum Allocation [2]

The first reviewed approach to the power optimization problem combines spectrum allocation with power allocation to optimize sum rate and minimize QoS violations. This is done since the two problems are overlapping in NOMA systems to determine which overlapping frequencies to allocate links to and how much power to allocate to each link to reduce the amount of noise.

A. CSI Input Tensor

A representation of the input CSI tensor, G , is described in Equation 2[2].

$$G = [G^1, \dots, G^K] \in \mathbb{R}^{N \times N \times K}$$

$$G^k = \begin{bmatrix} g_{11}^k & \dots & g_{1N}^k \\ \vdots & \ddots & \vdots \\ g_{N1}^k & \dots & g_{NN}^k \end{bmatrix} \quad (2)$$

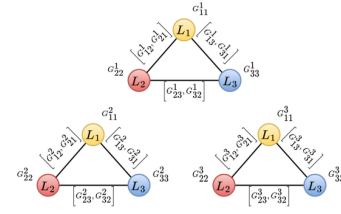


Figure 1: The Graph Creation Process for Joint Power and Spectrum Allocation [2]

The CSI input tensor G is made up of K matrices, one matrix for each resource block. Each matrix represents the interference between each link using that resource block. So g_{12}^4 represents the interference between links 1 and 2 using the fourth resource block. An element such as g_{11}^4 represents the direct link gain for the first link of the fourth resource block.

B. Graph Creation

The goal of graph creation for this architecture is to capture the interference between links for each resource block. One graph is created for each resource block so there are K graphs created. Each vertex of the graph represents a the direct link gain or g_{ii}^k . Each edge represents the interference between the two links using the k th resource block. An example the graph construction of using 3 links with 3 resource blocks is shown in Figure 1.

C. Message Passing

To process the graphs, the two functions shown in Equation 3[2] can represent the aggregate and update steps in the network.

$$m_{vu}^{t+1} = MLP_1^t(z_u^t, g_{uv}, g_{vu}) \quad (3)$$

$$z_v^{t+1} = MLP_2^t(z_v^t, \max\{m_{vu}^{t+1}, u \in N(v)\})$$

m_{vu}^{t+1} uses the vertex value u and the two edge values to the vertex v as input to a trainable MLP. z_v^{t+1} represents the value to update the vertex v with. The function aggregates each node using the function m_{vu}^{t+1} then takes the max value of the aggregated values. To update the vertex v the maximum aggregated value and the vertex v are sent through a different MLP. Once the message passing layers are complete, the vertex values in each graph are concatenated together creating K embeddings, one for each resource block. The embeddings are then used as input to a convolutional neural network (CNN).

D. CNN Layer and Output

The CNN layer is first responsible for performing a regular convolution then splitting the output to the

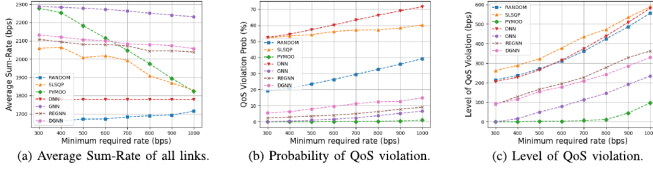


Figure 2: Power Control/Spectrum Allocation Results [2]

power vector and the resource block allocation matrix. One vector of length N is taken from the output to represent the power vector and a sigmoid function is applied. For the resource block allocation matrix, the rest of the output is used. Softmax is applied for each vector in the matrix representing the probabilities of using each resource block to determine which link will use which resource block. For a loss function, the authors use a standard mean squared error function as well as a loss function that ensures a link only uses as much capacity as needed so resources can be allocated to other links.

E. Results

Comparisons to different evaluation models are shown in Figure 2, the three to pay attention to are random, deep neural network (DNN), the author's approach with supervised training before being deployed (GNN), and the author's approach with only unsupervised learning after being deployed (DGNN).

For average sum rate, GNN is able to keep a large advantage over all other methods even while increasing the minimum required rate beating the next best method by about 200 bps. For QoS violation probability, GNN is beaten by PYMOO by a relatively low margin of around 5%. A noticeable observation from this graph is that DNN has a higher QoS violation probability compared to using a random scheme which showcases a DNNs inability to learn the power allocation problem. For the level of QoS violation GNN has the second lowest violation level also beaten by PYMOO.

Even though GNN breaks the QoS more than PYMOO the average sum rate is much higher using GNN and has a much lower execution time making the author's implementation a viable option for power allocation optimization [2].

III. Unfolding WMMSE [4]

The original WMMSE algorithm to solve the power optimization problem is a relatively slow algorithm to use in real time systems. This is because WMMSE is an iterative algorithm that has a slow convergence time to find optimal parameters. The authors of Unfolding WMMSE (UWMMSE) create an unfolded version of

WMMSE that parameterizes itself using GNNs while still preserving the nature of WMMSE. To do this the authors equate one iteration of WMMSE to one layer which contains equivalent equations as WMMSE which learn their parameters with GNNs. This both increases speed and sum rate achieved by UWMMSE. The layered architecture also limits the number of iterations UWMMSE can take dependent on the number of layers, K , while WMMSE continues to run iterations until a final condition is found making it slower if a final condition takes many iterations to find.

A. UWMMSE Architecture

The authors make a point that their architecture can be used with any permutation invariant message passing structure such as sum/mean as well as any graph structure so they do not focus on those two aspects. The architecture takes a layered approach with each layer using the output from the previous layer, $v^{(k-1)}$, to compute $v^{(k)}$ which is shown in Equation 4[4] where Ψ is a GNN, h_{ii} is a direct link, and h_{ij} is an indirect link.

$$\begin{aligned}
 a^k &= \Psi(H; \Theta_a^k) b^k = \Psi(H; \Theta_b^k) \\
 u_i^k &= \frac{h_{ii} v_i^{(k-1)}}{\sigma^2 + \sum_j h_{ij}^2 v_j^{(k-1)} v_j^{(k-1)}} \\
 w_i^k &= \frac{a_i^k}{1 - u_i^k h_{ii} v_i^{(k-1)}} + b_i^k \\
 v_i^k &= \alpha \left(\frac{u_i^k h_{ii} w_i^k}{\sum_j h_{ji}^2 u_j^k v_j^k w_j^k} \right)
 \end{aligned} \tag{4}$$

The key takeaways from these equations is that it uses the output from the previous layer, v_i^{k-1} , uses a GNN Ψ to find the parameters, and wants to maximize the direct link rate h_{ii} in the numerator while decreasing the indirect interference, h_{ji} in the denominator.

B. Results

For testing purposes the authors use a message passing scheme with learnable weights during the aggregation and update phases similar to a MLP. The authors do not elaborate on a graph structure only stating that they use the CSI information to construct the graph showcasing that UWMMSE can work with any input graph structure. The results are shown below in Figure 3.

UWMMSE is able to achieve both a higher sum-rate as well as a quicker test time, 2 ms compared to 16 ms. UWMMSE even achieves a quicker test time compared to using a MLP. Since the goal of UWMMSE is to reduce the test time of WMMSE and can achieve a higher sum rate, UWMMSE is viable to use to power allocation optimization [4].

Algorithm	Training time (min)	Test sum-rate	Test time (msec)
WMMSE [14]	-	82.94	16
Tr-WMMSE	-	76.49	1.0
MLP [23]	0.5	78.17	3.2
REGNN [25]	15	57.92	2.5
IGCNet [26]	5	55.30	3.0
UWMMSE	15	83.21	2.0

Figure 3: UWMMSE Results [4]

IV. Air-MPNN and Air-MPRNN [5]

A core problem with implementing power allocation estimation with GNNs is the overhead to message pass between vertices that represent direct links in real time to keep the distributed nature of a GNN so it can scale with network size. To fix this problem the authors use a method from over-the-air computation (AirComp) which can superimpose multiple signals together to calculate operations such as the sum, mean, and max of those signals all over the air [6]. The authors use AirComp to reduce the channel estimation overhead, a pilot signal used to estimate the CSI value held by each vertex when channel states change, and message passing overhead between devices when running the GNN.

A. Graph Creation

The graph creation used by the authors of Air-MPNN attempt to capture direct links and interference caused by other links to those direct links. The graph is a fully connected graph, which is important for the the Air-MPNN method. Each vertex representing a direct link i , h_{ii} , and each edge, h_{ij} , represents the interference between the direct link i and direct link j . The graph creation process is similar to the graph creation process used in the Joint Wireless Power Control and Spectrum Allocation study shown in Figure 1.

B. MPNN

MPNN is an already existing method to perform message passing before Air-MPNN which follows the aggregate then update pattern. MPNN first estimates the channel states by sending a pilot signal along every link resulting in $O(K^2)$ pilot signals for K links.

The large overhead occurs during the aggregation phase and since the graph is fully connected, every vertex broadcasts its message to every other vertex. Since the message being sent by each vertex is different, each broadcast must happen one-by-one which increases overhead. Once each vertex gathers the broadcast data from all the other vertices, the vertex can update its own value then the next GNN layer can

resume starting with aggregation again. This method results in the message passing signal overhead to be $O(NK)$ since a broadcast is used where N is the number of layers in the GNN. The channel estimation and message passing overhead are shown in Figure 4.

C. Air-MPNN

Air-MPNN uses aggregation over the air to reduce the channel estimation overhead and solve the one-by-one transmission problem in MPNN. When estimating the channel, Air-MPNN only sends pilot signals, p_i , along the direct links instead of every combination of interference links. This causes the channel estimation overhead to only be $O(K)$ compared to $O(K^2)$ in MPNN. Each link will learn the channel estimation of other links in the next step during aggregation.

During aggregation, when each node broadcasts its message, aggregation over the air is used so each vertex can broadcast simultaneously instead of one-by-one which reduces overhead. Aggregation over the air superimposes the signals to be added together while traveling through the air. This is shown in the aggregation over the air equation, $\tilde{a}_i^n$, shown in Equation 5[5]. The update function that uses the aggregated value includes learnable parameters. This aggregation function includes the pilot signal from the vertex's neighbor, p_j , so each vertex can get information about each pilot signal since pilot signals were only sent along direct links during channel estimation.

$$\tilde{a}_i^n = \text{sum}(\tilde{p}_j^n |h_{ji}|^2, j \in N(i)) \quad (5)$$

Since a broadcast still occurs, the number of signals sent is still the same, $O(NK)$, except the broadcasts can occur simultaneously. The channel estimation and message passing overhead are shown in Figure 4.

D. Air-MPRNN

Air-MPRNN combines a recurrent neural network (RNN) with Air-MPNN. This allows Air-MPRNN to predict channel changes overtime by using the previous iteration to make a prediction. Since temporal information about a link's state is used, the periodic check of a link's state from channel estimation is removed. This removes the channel estimation overhead completely. Air-MPRNN uses the same message passing technique as Air-MPNN so the signal overhead is $O(NK)$.

E. Results

The results are shown in Figure 5. It can be seen that both Air-MPNN and Air-MPRNN both achieve a much lower overhead ratio as well as a higher sum-rate. The training time for a Air-MPRNN is much higher due to the iterative nature of RNNs. The large decrease in

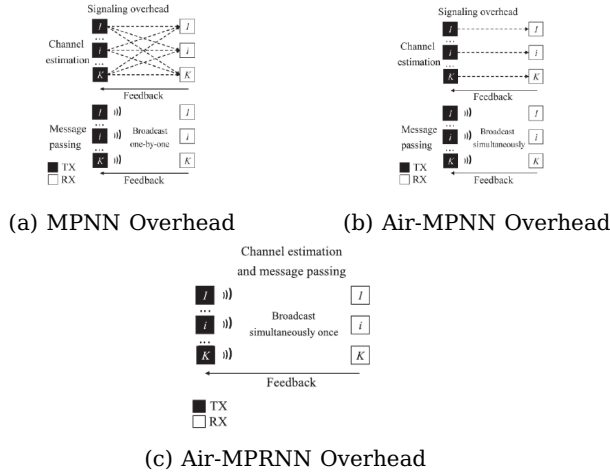


Figure 4: MPNN, Air-MPNN, and Air-MPRNN Overheads [5]

Algorithm	Training time (s)	Sum-rate (bps/Hz)	Overhead ratio $\frac{N_O}{N_S}$
EPA	-	74.17	0
WMMSE	-	78.88	13.3%
MPNN	14.84	67.26	23.3%
Air-MPNN	16.41	84.80	2.7%
Air-MPRNN	54.53	85.76	0.7%

Figure 5: Air-MPNN and Air-MPRNN Results

needed overhead makes Air-MPRNN a viable enhancement to MPNN [5].

V. Comparisons

A. Graph Creation

Joint power control/spectrum allocation and Air-MPNN/Air-MPRNN create a graph based on the direct links as vertices with edges representing the interference caused by other links. In the context of power allocation, which attempts to reduce the total amount of interference between non-orthogonal signals, this is a good representation. UWMMSSE does not specify an exact graph creation method only that they use the CSI information since their architecture can use any graph creation process as input.

B. Message Passing

All three reviewed papers use a similar message passing technique using learnable weights either in their aggregate or update functions. The unique case is Air-MPNN and Air-MPRNN which aggregates their values over the air using a method similar to AirComp [6] through a summation.

C. Architecture

All three reviewed papers use GNNs within their architectures. Air-MPRNN also uses temporal information in its architecture. Joint power control/spectrum

allocation uses multiple graphs for each resource block while UWMMSSE and Air-MPNN use one graph in their architecture.

D. Sum Rate Increases and Speed Optimizations

All three reviewed papers are able to increase the sum rate. Power control/spectrum allocation does not include any speed optimizations besides using GNNs, instead using multiple graphs which increases the time needed to run it. UWMMSSE focuses on optimizing for speed compared to WMMSE by using a GNN combined with the WMMSE formula. Air-MPNN/Air-MPRNN include speed optimizations by decreasing the amount of channel estimation and message passing overhead.

This means power control/spectrum allocation could be used for systems that have available delay time while running in a real time scenario while UWMMSSE and Air-MPNN/Air-MPRNN should be used when reducing delay is important. Combinations of these have not been explored in this review such as using UWMMSSE as the main architecture while introducing aspects from Air-MPRNN to decrease signal overhead.

VI. Conclusion

In conclusion, GNNs can capture network topology data as well as run quicker compared to other methods. This allows GNNs to make quick and accurate predictions about power allocation in NOMA systems allowing multiple signals to use the same frequency bands increasing the throughput of the system. The reviewed papers cover both increasing the sum rate and optimizing the speed of the systems which are important for real time systems.

References

- [1] F. Saggese, A. Abrardo, M. Moretti, and M. Morelli, "Low complexity wmmse power allocation in noma-fd systems," *arXiv preprint arXiv:1905.06681*, 2019.
- [2] M. Marwani and G. Kaddoum, "Graph neural networks approach for joint wireless power control and spectrum allocation," *IEEE Transactions on Machine Learning in Communications and Networking*, 2024.
- [3] Y. Shen, J. Zhang, S. Song, and K. B. Letaief, "Graph neural networks for wireless communications: From theory to practice," *IEEE Transactions on Wireless Communications*, vol. 22, no. 5, pp. 3554–3569, 2022.
- [4] A. Chowdhury, G. Verma, C. Rao, A. Swami, and S. Segarra, "Unfolding wmmse using graph neural networks for efficient power allocation," *IEEE Transactions on Wireless Communications*, vol. 20, no. 9, pp. 6004–6017, 2021.
- [5] Y. Gu, C. She, Z. Quan, C. Qiu, and X. Xu, "Graph neural networks for distributed power allocation in wireless networks: Aggregation over-the-air," *IEEE Transactions on Wireless Communications*, vol. 22, no. 11, pp. 7551–7564, 2023.
- [6] B. Nazer and M. Gastpar, "Computation over multiple-access channels," *IEEE Transactions on information theory*, vol. 53, no. 10, pp. 3498–3516, 2007.